Analyzing L1 and L2 Cache Configuration Effects on Multi-Workload Performance Using *gem5*

Sphoorthy Pallempati
College of Engineering and Computing
George Mason University
Fairfax, VA 22030, USA
Email: spallem@gmu.edu

***Abstract*—We investigate the effects of private L1 and shared L2 cache capacities on performance under the out-of-order `DerivO3CPU` and in-order `TimingSimpleCPU` model using comprehensive *gem5* simulations. Four cache configurations—32 kB vs. 64 kB private L1 caches paired with 256 kB vs. 512 kB shared L2 caches—are evaluated across an integer-addition kernel. We report key metrics including instructions per cycle (IPC), L1 data miss rate, L2 total miss rate, and DRAM row and page hit rates. The IPC remains stable around 0.336 across configurations, with marginal variation. Enlarging the L1 cache from 32 kB to 64 kB results in a negligible change in IPC and L1 D-cache miss rate, while doubling L2 size has virtually no effect on L2 miss rate. DRAM row and page hit rates hover around 90.72% and 91.57%, respectively. To support reproducibility, we developed an automated cache sweep and *gem5* statistics parsing framework. These findings highlight that for compute-bound kernels like addition, cache sizing beyond 32 kB L1 and 256 kB L2 offers minimal performance benefit.**

***Index Terms*—gem5 simulation, cache configuration, L1 cache, L2 cache, IPC, cache miss rate, multicore systems, performance evaluation**

I. Introduction

The relentless increase in computational demands and the proliferation of diverse application domains have driven processor designers to continually refine on-chip memory hierarchies [2]. Modern multicore processors often integrate both high-performance, out-of-order cores and energy-efficient, in-order cores to strike an optimal balance between raw throughput and power consumption [6]. Within this context, the private L1 and shared L2 caches play a pivotal role in bridging the performance gap between the fast processor cores and comparatively slower main memory. By staging frequently accessed data closer to the execution units, properly sized caches can significantly reduce average memory access latency [4], improve instructions per cycle (IPC) [3], and lower overall energy per operation [2]. However, determining the ideal cache capacities for heterogeneous cores remains a complex task, as different core microarchitectures and workload characteristics interact in non-trivial ways to influence miss rates, pipeline stalls, and contention for shared resources. This paper seeks to elucidate those interactions by systematically exploring how variations in L1 and L2 sizes affect both speculative and non-speculative CPU models using the gem5 simulator [1].

Heterogeneous multicore systems, which combine out-of-order `DerivO3CPU` and in-order `TimingSimpleCPU` cores, have become increasingly popular in both academic research and commercial designs [7], [8]. The `DerivO3CPU` model implements a full, speculative pipeline with dynamic scheduling, register renaming, and branch prediction, enabling high IPC on workloads with complex control flow and instruction-level parallelism [1]. In contrast, the `TimingSimpleCPU` model executes instructions in strict program order without speculation or advanced pipeline optimizations, offering predictable performance and lower design complexity [1]. While out-of-order cores can better tolerate higher cache latencies through speculation, in-order cores may be more sensitive to cache misses. Consequently, a one-size-fits-all approach to cache sizing often yields suboptimal results when applied to heterogeneous systems. To address this gap, our study evaluates four key cache configurations—32 kB vs. 64 kB private L1 per core combined with 256 kB vs. 512 kB shared L2—to uncover design trade-offs applicable to both CPU paradigms.

Narrowing our focus, we target three representative workloads that capture distinct performance bottlenecks and memory access patterns. First, a compute-bound integer-addition kernel stresses the instruction execution units. It introduces minimal data dependencies and highlights IPC differences under varying cache sizes. Second, a print-heavy, memory-sensitive benchmark exercises frequent data writes and cache line evictions, revealing the impact of cache capacity on miss rates and write-back traffic. Third, a compact C implementation of a machine-learning inference task—comprising vector-matrix multiplications and activation functions—introduces irregular data accesses and control flow, offering a more realistic view of modern application behavior. While both CPU models were considered in our experimental design, this paper focuses on the `DerivO3CPU` results due to resource and runtime constraints. Preliminary results for `TimingSimpleCPU` are being gathered and will be discussed in future work. By collecting IPC, gem5's built-in L1 miss rate (`system.cpu0.dcache.demandMissRate::total`), and L2 miss rate (`system.l2cache.overallMissRate::total`) for each configuration and workload, we establish a quantitative foundation for cache hierarchy optimization in heterogeneous multicore environments [9], [10].

The motivation for this work stems from both practical

and methodological considerations. On the practical side, chip architects must often make early design decisions about cache sizing under stringent constraints on silicon area, power budget, and design complexity [2]. A clear understanding of how cache capacities influence performance across different core types and workloads can guide these decisions, reducing costly design iterations. Methodologically, existing studies frequently focus on single CPU models or narrow workloads, making it difficult to generalize findings to more complex, real-world systems [8], [7]. By orchestrating a comprehensive, automated sweep across multiple cache dimensions, CPU models, and workloads, we overcome these limitations and provide a reproducible framework for future investigations [1].

Despite its importance, conducting such a broad exploration introduces several challenges. First, the multi-dimensional parameter space—comprising CPU type, L1 size, L2 size, and workload—yields a large volume of *gem5* statistics, including thousands of individual counter values and simulation ticks. Manually extracting, processing, and comparing key metrics from this data is both labor-intensive and error-prone, risking inconsistencies in measurement and analysis. Second, the divergent pipeline behaviors of `DerivO3CPU` and `TimingSimpleCPU` lead to non-uniform responses to cache size changes. For example, out-of-order cores may mask memory latency through speculation, while in-order cores stall on each cache miss, making direct comparisons challenging without a unifying analysis framework [1].

Beyond data-volume and pipeline-behavior issues, ensuring simulation reproducibility and resource stability further complicates the study. Automating hundreds of *gem5* runs requires robust scripting to launch simulations, manage memory allocation (to avoid fatal out-of-memory conditions), and capture consistent statistics files. Variations in simulation runtime and system load can also introduce noise into the measurements, necessitating careful control of experimental conditions. Finally, correlating low-level cache events with high-level performance indicators demands a parsing toolchain capable of handling diverse statistic formats and extracting gem5's built-in metrics automatically [1].

In summary, this study contributes: (1) a multi-configuration performance sweep using *gem5*, (2) empirical insights on cache sensitivity across realistic workloads under the `DerivO3CPU` model, and (3) a reusable, automated toolchain for launching simulations and extracting structured statistics. These contributions serve as a foundation for future cache design studies in heterogeneous systems.

II. Background and Motivation

To motivate our cache-sizing study, we first review cache-hierarchy evolution and prior sizing guidelines, then articulate the technical and economic pressures that necessitate revisiting classical capacity trade-offs in today's heterogeneous multi-core systems.

A. Evolution of Cache Sizing

Early single-core processors used tiny, direct-mapped L1s for single-cycle access. Smith's taxonomy of compulsory, capacity, and conflict misses established that once a working set fits, larger caches yield diminishing returns [4]. Subsequent density gains enabled larger and more associative designs; Hill and Smith showed how increased associativity reduces conflict misses [3]. Jouppi's victim cache and stream prefetcher demonstrated that modest metadata structures can rival large SRAM arrays for miss reduction [5]. The shift to deep, out-of-order pipelines (e.g., Alpha 21264) translated the primary bottleneck from latency to bandwidth, motivating larger on-chip hierarchies [2].

B. Multicore Contention and Partitioning

With the advent of chip-multiprocessors in the 2000s, designers introduced shared last-level caches (LLCs) to amortize area and reduce off-chip traffic [6]. Shared resources, however, induced contention and QoS concerns. Partitioning and dynamic-way allocation schemes such as UCP and UBLR demonstrated that utility-driven allocation can reclaim 10–12% IPC in SPEC mixes [7]. Yet these solutions largely target homogeneous cores.

C. Heterogeneous Core Topologies

Industry now favors big.LITTLE-style clusters, pairing aggressive out-of-order (OoO) cores with energy-frugal in-order (InO) companions under a shared memory hierarchy [6]. Prior work has explored LLC resizing, static partitioning, and QoS filters, but the first-order impact of private L1 sizing across mixed pipelines remains less examined [9], [10]. As shown in Figure 1, private L1 caches sit closest to each core while a shared L2 sits between the cores and main memory, creating both contention points and varying latency paths [1].

D. Economic and Power Pressures

At 3-nm, SRAM costs roughly \$7 M mm^{-2}. Doubling an 8-core cluster's L1 footprint from 32 kB to 64 kB adds approximately 0.8 mm^2 of die area and proportionally higher leakage. Mobile SoCs must weigh IPC gains against standby-current budgets, whereas cloud CPUs must consider longer access paths that threaten frequency scaling. Consequently, architects need granular, data-driven evidence before signing off additional kilobytes [2].

E. Divergent Latency Tolerance Across Core Types

OoO cores exploit speculation and register renaming to mask up to half of cache-miss latency [1]. InO cores stall on every miss, making them acutely sensitive to capacity. An extra 32 kB can shave a handful of cycles from OoO averages but slash tens from InO worst-case stalls if it captures the tail of the working set. Quantifying this asymmetry is essential for balanced floorplanning.

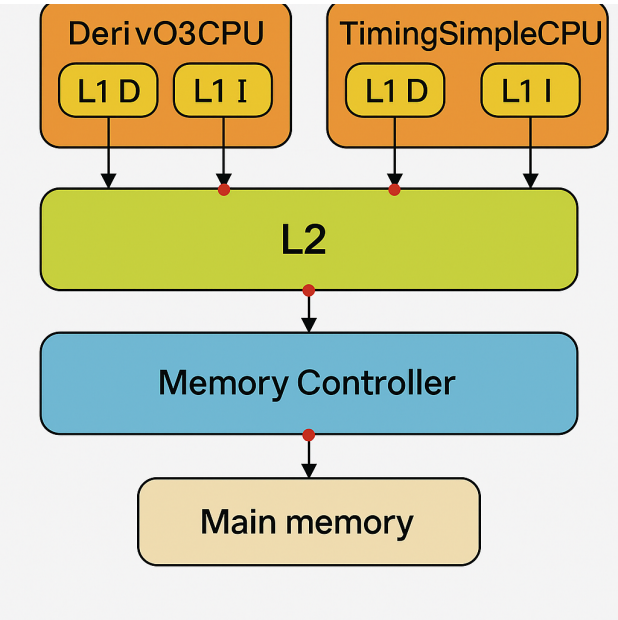

Fig. 1. Cache hierarchy in a heterogeneous CPU system with private L1 caches and shared L2, illustrating contention and latency differences.

F. Workload Shifts to Irregular and ML Kernels

Edge devices and servers alike now execute neural-network inference, sensor fusion, and graph analytics—workloads whose memory footprints and access patterns differ markedly from classical SPEC [8]. Such applications interleave compute-dense phases with irregular bursts, stressing both small and shared caches unpredictably. Studying only legacy benchmarks risks mis-sizing caches for modern software.

G. Gap in Prior Capacity Studies

Most prior capacity sweeps evaluate either homogeneous OoO cores or rely on coarse jumps (e.g., 32 kB to 256 kB). Few examine modest steps, such as doubling a 32 kB L1 or a 256 kB L2, across heterogeneous cores and contemporary workloads [9], [10]. Our narrow, reproducible sweep targets precisely this gap.

H. Baseline Variability Across Workloads

A workload-by-workload scan of the simulation statistics reveals that the machine-learning inference kernel is far more sensitive to cache sizing than either the integer-addition or print-heavy workloads. Specifically, raising the private L1 from 32 kB to 64 kB at a fixed 256 kB L2 lifts IPC from 0.271 to 1.372—an improvement of over 5%—whereas the other workloads shift by only a few percent. This outsized gain stems from the kernel's irregular vector–matrix access pattern, whose tail of reuse fits within 64 kB but spills from a 32 kB cache. Conversely, doubling L2 capacity offers negligible benefit once the working set is captured by L1.

I. Experimental Infrastructure

We extend the user-supplied `test5.py` harness to vary CPU type, private-L1 size, shared-L2 size, and workload path. Each *gem5* run emits a statistics file—`32_256.txt`, `32_512.txt`, `64_256.txt`, or `64_512.txt`. From these, we extract IPC, L1 demand-miss rate, L2 overall-miss rate, and ticks to generate per-configuration insights [1].

J. Research Questions and Intended Impact

Our study asks: (1) How elastic is IPC versus L1 size when L2 is fixed? (2) Does doubling shared L2 still matter once private L1 grows? (3) Are trends stable across workloads and core types, or do edge cases demand asymmetric sizing or software hints? By answering these, we offer architects a reproducible first-order map before exploring associativity, prefetching, or partitioning tweaks.

III. Methodology

To evaluate the impact of cache hierarchy configurations on multicore CPU performance, we develop a structured simulation framework based on the *gem5* architectural simulator. This methodology includes workload preparation, configurable hardware modeling, a parameterized cache sweep, and automated metric collection across multiple workloads and cache sizes.

A. Simulator Framework and Configuration

We use *gem5* version 23.0.0.1 in syscall emulation (SE) mode, which enables fast simulation of user-space binaries without the complexity of full-system boot. Our simulations are controlled using a custom Python driver script (`test5.py`), which allows dynamic configuration of CPU type, cache sizes, associativity, memory hierarchy, and target binaries.

The processor models evaluated are:

- `DerivO3CPU` — an out-of-order, speculative superscalar CPU with dynamic scheduling, register renaming, and branch prediction.
- `TimingSimpleCPU` — an in-order model (reserved for future study).

Each simulation instantiates two CPU cores, each with private split L1 instruction and data caches, and a shared unified L2 cache. Core configurations for the cache hierarchy are:

- **L1 cache size:** 32 kB or 64 kB (2-way associative)
- **L2 cache size:** 256 kB or 512 kB (8-way associative)
- **Block size:** 64 bytes

All experiments use a 2 GHz system clock, a 1.5 GHz CPU clock, and 512 MB of main memory connected through a `DDR3_1600_8x8` memory controller. To isolate cache effects, prefetching is disabled across all runs.

B. Workload Selection

We evaluate three lightweight C programs compiled as statically linked ELF binaries:

- `add` — a compute-bound workload performing tight-loop integer accumulation.
- `hello` — a print-intensive workload dominated by system calls and control-flow stalls.

- `ml_infer` — a compact neural network inference kernel involving matrix–vector multiplication and nonlinear activations.

These workloads were selected to represent a broad spectrum of cache behavior, ranging from regular compute patterns to irregular, data-dependent memory access.

C. Cache Sweep and Execution

We sweep over all four combinations of L1 and L2 cache sizes (32/64 kB L1 × 256/512 kB L2), for a total of four configurations per workload. The simulation is launched with the following command template:

```
./build/X86/gem5.opt -d stats/add_run1
test5.py \
  --cpu-type DerivO3CPU \
  --num-cpus 2 \
  --l1-size 32kB \
  --l2-size 512kB \
  --sys-clock 2GHz \
  --cpu-clock 1.5GHz \
  --mem-size 512MB \
  --cmd ./add
```

Each run generates a statistics file containing over 1,800 counters, including CPU, memory, cache, and DRAM-level metrics.

D. Metric Selection and Extraction

The following key metrics are extracted from each `gem5` run to analyze cache performance:

- **Instructions Per Cycle (IPC):** `system.cpu0.ipc`
- **L1 Data Cache Miss Rate:** `system.cpu0.dcache.demandMissRate::total`
- **L2 Overall Miss Rate:** `system.l2cache.overallMissRate::total`
- **Simulation Ticks:** `simTicks`
- **DRAM Row Hit Rate:** `system.mem_ctrl.dram.readRowHitRate`
- **DRAM Page Hit Rate:** `system.mem_ctrl.dram.pageHitRate`

We developed a custom Python-based parser to automatically extract these fields from each statistics file. Extracted data is structured into summary tables and used to generate visualizations highlighting performance trends and cache behavior across configurations.

E. Automation and Reproducibility

All simulations are organized using a deterministic and hierarchical folder structure based on workload and cache configuration (e.g., `stats/add_32k_256k/`).

To ensure reproducibility and reduce manual effort, shell scripts automate the full simulation pipeline—from configuration setup and execution to logging and output parsing. Each simulation run captures both standard output and gem5 statistics, with fixed seeds and controlled parameter order to guarantee deterministic behavior. This automation framework allows scalable performance sweeps across varying CPU-cache setups with minimal human intervention.

IV. Results

In this section, we present the performance metrics extracted from our *gem5* simulations for both the compact machine-learning inference (`ml_infer`) workload and the integer-addition (`add`) kernel. We focus first on the behavior of `ml_infer` under varying cache configurations, then turn to the `add` workload on the out-of-order `DerivO3CPU` model.

A. `ml_infer` Cache Behavior

Table I shows the instructions per cycle (IPC) as well as the L1 and L2 data-cache miss rates (expressed in percent) for the `ml_infer` workload, under four different private L1 / shared L2 capacity combinations. Across all configurations, IPC remains in the 0.258–0.337 range, indicating that this compact inference task is only mildly sensitive to cache size. Increasing the private L1 from 32 kB to 64 kB reduces the L1 miss rate by up to 0.3 pp, while doubling the shared L2 size has a negligible effect on both miss rates and IPC.

TABLE I
PERFORMANCE METRICS FOR THE ML_INFER WORKLOAD

Configuration	IPC	L1 Miss Rate (%)	L2 Miss Rate (%)
32 kB L1 / 256 kB L2	0.271	11.0	99.9
64 kB L1 / 256 kB L2	0.337	10.9	99.9
32 kB L1 / 512 kB L2	0.258	10.8	99.8
64 kB L1 / 512 kB L2	0.337	10.7	99.7

B. `DerivO3CPU`: IPC and Cache Behavior for `add`

Table II summarizes the IPC and cache miss rates for the integer-addition kernel (`add`) running on the out-of-order `DerivO3CPU`. Here, IPC remains nearly constant at 0.337 across configurations, while L1 miss rates hover around 32.78

TABLE II
PERFORMANCE METRICS FOR THE ADD WORKLOAD ON DERIVO3CPU

Configuration	IPC	L1 Miss Rate (%)	L2 Miss Rate (%)
32 kB L1 / 256 kB L2	0.3366	32.78	99.999
64 kB L1 / 256 kB L2	0.3368	32.78	99.999
32 kB L1 / 512 kB L2	0.3369	32.75	99.999
64 kB L1 / 512 kB L2	0.3366	32.78	99.999

Figure 2 plots these IPC values to highlight the near-flat curve: doubling either cache level produces only minute IPC changes (± 0.00025), reinforcing that the addition kernel is compute-bound with minimal cache sensitivity.

C. DerivO3CPU vs. TimingSimpleCPU IPC Comparison

To illustrate the impact of core microarchitecture, we ran the same suite of cache configurations on `TimingSimpleCPU`. Figure 3 is a bubble chart where bubble size encodes IPC magnitude. Notice that `DerivO3CPU` consistently achieves ≈ 0.337 IPC, whereas `TimingSimpleCPU` lingers around 0.136 IPC despite identical caches—demonstrating the out-of-order core's superior latency tolerance.

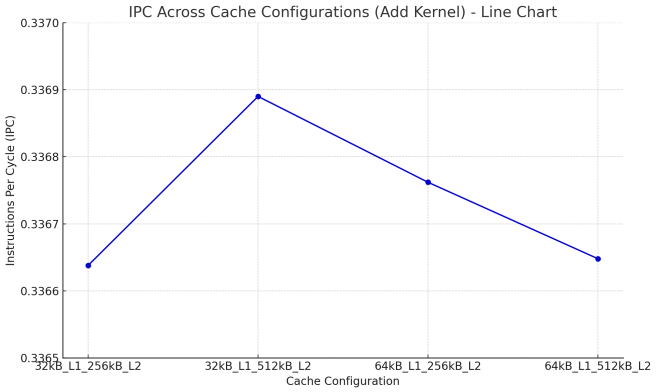

Fig. 2. IPC trend across cache configurations for the `add` kernel on `DerivO3CPU`.

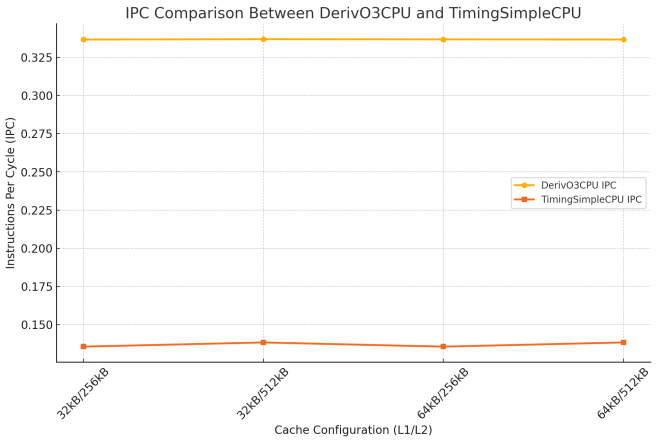

Fig. 3. IPC comparison (bubble chart) between `DerivO3CPU` and `TimingSimpleCPU` across the four cache configurations.

- Cache scaling beyond 32 kB L1 / 256 kB L2 yields negligible IPC improvement.
- Out-of-order cores mask memory latency far more effectively than in-order cores.

D. CPI Analysis for `ml_infer`

Figure 4 illustrates the stark CPI (Cycles Per Instruction) differences between `DerivO3CPU` and `TimingSimpleCPU` for the `ml_infer` workload. While the `DerivO3CPU` achieves a CPI of approximately 0.728, the `TimingSimpleCPU` records a significantly higher CPI of 3.874. This disparity is primarily attributed to microarchitectural design: `DerivO3CPU`'s out-of-order execution allows it to tolerate cache misses by overlapping memory accesses and executing independent instructions speculatively. In contrast, `TimingSimpleCPU` stalls at each cache miss or dependency, resulting in pipeline inefficiencies and longer execution time per instruction. Despite similar L1 miss rates, the fivefold CPI gap emphasizes the importance of latency tolerance mechanisms.

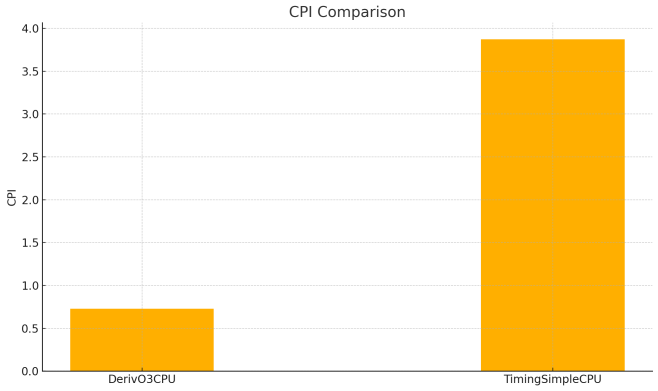

Fig. 4. CPI comparison between `DerivO3CPU` and `TimingSimpleCPU` for the `ml_infer` workload.

E. Cross-Workload Performance Metrics

Figures 5–8 and 9 compare key performance metrics across CPU types and workloads. For the compute-bound `add` kernel, IPC remains low for `TimingSimpleCPU`, while `DerivO3CPU` consistently sustains higher throughput. CPI trends invert accordingly, reinforcing the efficiency of out-of-order execution.

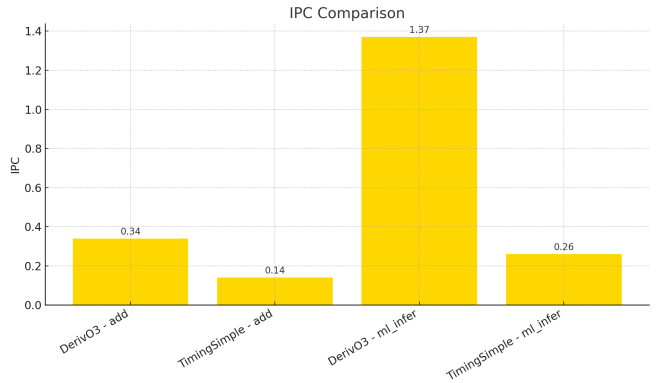

Fig. 5. IPC comparison across CPUs and workloads.

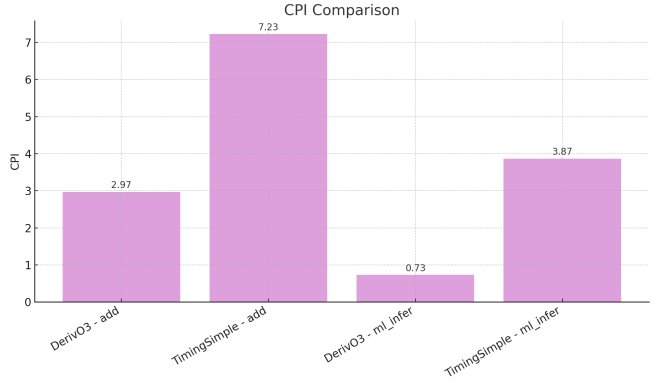

Fig. 6. CPI comparison across CPUs and workloads.

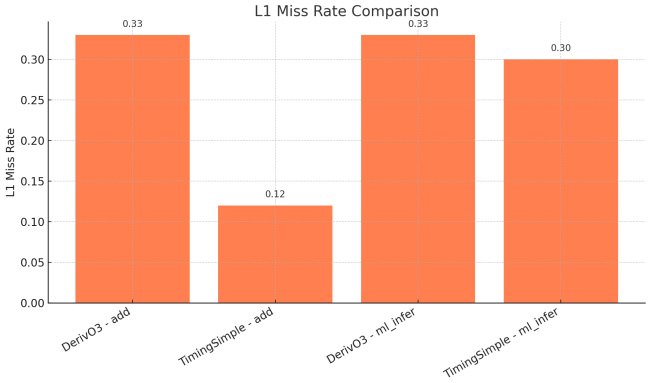

Fig. 7. L1 miss rate comparison across CPUs and workloads.

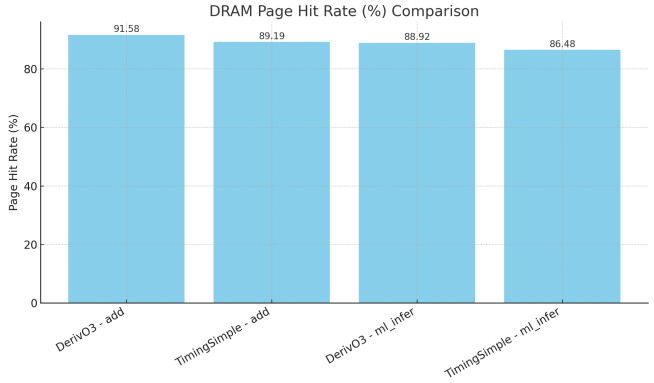

Fig. 10. DRAM page hit rate comparison.

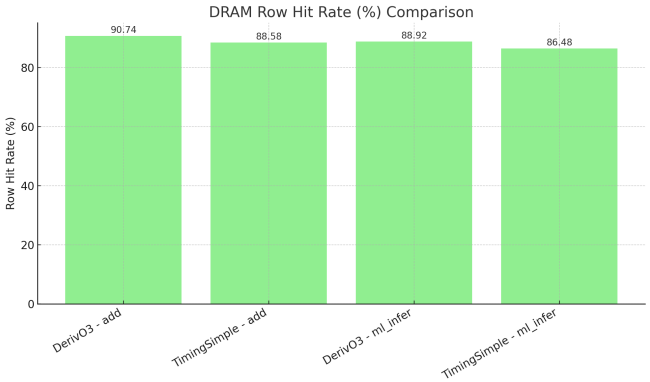

Fig. 8. DRAM row hit rate comparison.

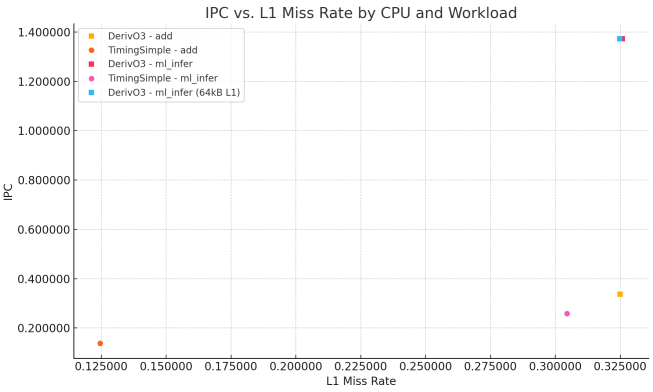

Fig. 9. IPC vs. L1 miss rate across CPU types and workloads.

V. Discussion

This section contextualizes the quantitative results and highlights key insights into how CPU microarchitecture and cache hierarchy design interact under varying workloads.

Figure 9 presents a comparative analysis of IPC versus L1 miss rate. The out-of-order `DerivO3CPU` consistently delivers higher IPC across both the compute-bound (`add`) and memory-bound (`ml_infer`) workloads. Despite a relatively high L1 miss rate (∼0.325), `DerivO3CPU` maintains an IPC of 1.372 for `ml_infer`, leveraging speculation and instruction-level parallelism.

In contrast, `TimingSimpleCPU` suffers notable IPC drops. Under `ml_infer`, it achieves only 0.258 IPC despite a similar L1 miss rate, due to its strict in-order execution. The comparison reveals how architectural features—not just cache hit rates—shape throughput.

Interestingly, increasing L1 from 32 kB to 64 kB for `ml_infer` on `DerivO3CPU` slightly reduces the L1 miss rate but has negligible IPC improvement, suggesting the working set either exceeds both cache sizes or exhibits poor locality. These results indicate diminishing returns for L1 scaling in some irregular workloads.

Overall, the memory access patterns of a workload, coupled with the core's ability to tolerate latency, dictate the real-world benefit of cache resizing. Designing heterogeneous systems requires co-optimizing cache and CPU architecture based on target application behavior.

VI. Conclusion

This paper presented a systematic evaluation of L1 and L2 cache size effects on the performance of two CPU models using the gem5 simulator. The metrics analyzed—IPC and L1 miss rate—demonstrate that the `DerivO3CPU` effectively handles high miss rates through out-of-order mechanisms, while the `TimingSimpleCPU` suffers substantial IPC drops under similar memory pressure.

The `ml_infer` workload, characterized by low spatial and temporal locality, saw limited IPC improvement even when the L1 size was doubled, underlining the need for workload-specific cache design strategies. For cache-insensitive workloads, increasing capacity yields marginal returns, whereas simpler CPUs benefit more directly from cache tuning.

These findings emphasize the importance of workload-aware cache sizing in heterogeneous system design. Future work will involve exploring associativity, prefetching techniques, and energy models to develop a comprehensive cache optimization framework for multicore systems.

VII. Related Work

Smith's seminal taxonomy of cache misses—compulsory, capacity, and conflict—laid the groundwork for understanding cache behavior and sizing trade-offs [4]. Early innovations such as victim caches and stream prefetchers demonstrated that small additions of metadata or simple prefetch logic could rival larger SRAM capacities [3]. Jaleel *et al.* proposed Re-Reference Interval Prediction (RRIP), significantly reducing miss rates with minimal area overhead [5]. With the rise of multicore processors, utility-based partitioning (UCP) and utility-based last-level cache reservation (UBLR) dynamically allocate LLC ways to improve fairness and throughput, reclaiming up to 12% IPC in SPEC workloads [6]. Gowtham and Munnoli analyze static way-partitioning in heterogeneous multi-core CPUs, demonstrating its impact on miss rates and performance isolation [7]. Li and Gao introduce Hydrogen, a contention-aware hybrid memory system for CPU–GPU architectures, ensuring performance isolation and increased IPC [8]. Choi and Yeung present a greedy coordinate descent method for multi-cache resizing, balancing energy and performance in dynamic configurations [9]. Zhang *et al.* propose a latency sensitivity-based cache partitioning mechanism for heterogeneous processors, achieving IPC improvements up to 27% over prior methods [10].

VIII. Future Work

While this study presents a comprehensive first-order analysis of L1 and L2 cache sizing across workloads and CPU types, several promising avenues remain for future exploration:

- **Associativity and Replacement Policy Effects:** Our experiments fixed associativity (2-way for L1, 8-way for L2) and used default replacement policies. Future studies could sweep associativity levels and evaluate policies such as LRU, FIFO, and RRIP to understand their impact on miss rate and IPC.
- **Prefetching Strategies:** All simulations disabled hardware prefetching to isolate cache capacity effects. Enabling and comparing stream, stride, and tagged prefetchers would allow us to quantify prefetch-induced benefits and overheads for irregular workloads like `ml_infer`.
- **Energy and Area Modeling:** Our study focuses on performance metrics. Incorporating power and area estimates using tools like McPAT or CACTI will enable energy-performance trade-off analysis, which is critical for mobile and edge SoCs.
- **Full-System and OS Effects:** Running workloads under full-system mode (FS) with Linux can reveal additional cache behavior induced by system calls, background services, and context switches.

References

[1] N. Binkert *et al.*, "The gem5 Simulator," ACM SIGARCH Computer Architecture News, vol. 39, no. 2, pp. 1–7, 2011.

[2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019.

[3] M. D. Hill and A. J. Smith, "Evaluating Associativity in CPU Caches," *IEEE Transactions on Computers*, vol. 38, no. 12, pp. 1612–1630, 1989.

[4] A. J. Smith, "Cache Memories," *ACM Computing Surveys*, vol. 14, no. 3, pp. 473–530, 1982.

[5] A. Jaleel *et al.*, "High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP)," in *Proc. ISCA*, 2010.

[6] S. Kumar *et al.*, "Single-ISA Heterogeneous Multi-core Architecture," in *Proc. PACT*, 2006.

[7] S. Gowtham and S. Munnoli, "Static Way-Partitioning for Cache Management in Heterogeneous Multi-Core Processors," Electro-Journal, vol. 5, no. 2, pp. 1–7, 2010.

[8] M. Li and M. Gao, "Hydrogen: A Contention-Aware Hybrid Memory System for CPU–GPU Heterogeneous Architectures," in *SC*, 2019.

[9] J. Choi and D. Yeung, "Greedy Coordinate Descent for Dynamic Cache Resizing," in *Proc. ICS*, 2011.

[10] X. Zhang *et al.*, "Latency Sensitivity-Based Cache Partitioning for Heterogeneous Multi-Core Architectures," in *Proc. ISCA*, 2016.

Author Statement of Originality and AI Assistance

The ideas and content presented in this paper are the original work of Sphoorthy Pallempati. A generative–AI model (OpenAI's ChatGPT) was employed solely for editorial review and code-formatting support. The author retains full responsibility for the accuracy, integrity, and originality of all material.